ABSTRACT

A fundamental limitation of current AI agents is their inability to learn complex skills on the fly at test time, often behaving like "clever but clueless interns" in novel environments. This severely limits their practical utility. To systematically measure and drive progress on this challenge, we first introduce the **Jericho Test-Time Learning (J-TTL)** benchmark. J-TTL is a new evaluation setup where an agent must play the same game for several consecutive episodes, attempting to improve its performance from one episode to the next. On J-TTL, we find that existing adaptation methods like reflection, memory, or reinforcement learning struggle. To address the challenges posed by our benchmark, we present **EvoTest**[1], an evolutionary test-time learning framework that improves an agent without any fine-tuning or gradients—by evolving the entire agentic system after every episode. EvoTest has two roles: the **Actor Agent**, which plays the game, and the **Evolver Agent**, which analyzes the episode transcript to propose a revised configuration for the next run. This configuration rewrites the prompt, updates memory by logging effective state–action choices, tunes hyperparameters, and learns the tool-use routines. On our J-TTL benchmark, EvoTest consistently increases performance, outperforming not only reflection and memory-only baselines but also more complex online fine-tuning methods. Notably, our method is the only one capable of winning two games (Detective and Library), while all baselines fail to win any.

1 INTRODUCTION

The pursuit of truly autonomous agents hinges on a critical human capability: the ability to learn "on the fly" (Maes, 1993; Franklin & Graesser, 1996). When faced with a new task, humans can attempt it, reflect on their successes and failures, formulate a better strategy, and try again. By contrast, most AI agents arrive at deployment with a fixed policy, behaving like "clever but clueless interns" that can execute instructions but cannot reform their own process from experience (Huang et al., 2024; Talebirad & Nadiri, 2023; Wang et al., 2024; 2025; Hou et al., 2023). This gap severely limits their reliability in dynamic settings. While the field acknowledges this problem, progress has been hampered by a lack of standardized testbeds designed specifically to measure an agent's capacity for rapid, in-session improvement (Zhou et al., 2023; Mialon et al., 2023; He et al., 2025a; 2024; 2026; Li et al., 2026; Yang et al., 2026; Sui et al., 2024a).

To address this, we first introduce the **Jericho Test-Time Learning (J-TTL)** benchmark, a new evaluation framework designed to systematically measure and drive progress in on-the-fly agent learning. The benchmark's core task is straightforward: an agent must play the same complex, text-based adventure game (Hausknecht et al., 2020) for a series of consecutive attempts ("episodes"). In

each episode, the agent interacts with the environment through a standard loop: it receives a textual observation of its surroundings (state), submits a natural-language command (action), and receives a numerical score change (reward). These games are difficult for LLM agents because they feature complex puzzles, long-range planning, sparse rewards (many critical actions yield no points), and irreversible consequences (a single wrong move can make the game unwinnable). The agent's goal is structured at two levels: 1) The *Episodic Goal*: Maximize the final score within a single playthrough. 2) The *Learning Goal*: Play the same game repeatedly and progressively increase its final score from one episode to the next, using only the experience gathered within that single session.

The J-TTL benchmark starkly reveals the inadequacies of existing adaptation paradigms. Consider a simple but critical failure in the game *Detective*: an agent gets stuck in a navigation loop by repeatedly attempting an invalid action, such as `GO WEST`, which the game rejects with `"You can't go that way."` This seemingly simple failure reveals deep flaws in current adaptation methods: A **Static** agent has no learning mechanism and will likely repeat this error in every episode, leading to a flat, low-scoring performance. An **SFT (online)** agent will have no good data to learn from in this failed episode. It is trapped because it cannot generate the very data it needs to improve. An **Reinforcement Learning (RL)(online)** agent receives a `reward=0` for the invalid move, which is a weak signal in a sparse-reward environment. A single update based on this noisy signal is insufficient to correct the policy, demonstrating a failure of credit assignment. Methods based on **reflection**, such as Reflexion (Shinn et al., 2023), modify the agent's prompt with summaries of past failures. While useful, it does not alter the agent's core decision-making logic or its use of tools. Similarly, advanced **memory systems** (Packer et al., 2023; Zhong et al., 2024) improve an agent's ability to recall information but do not teach it how to act differently. On the other end of the spectrum, RL and online fine-tuning are fundamentally ill-suited for the test-time learning setting. These methods are too slow and data-inefficient for the rapid learning J-TTL demands. To meet the challenge posed by our benchmark, we introduce **EvoTest, an evolutionary test-time learning framework designed for rapid, holistic adaptation without fine-tuning.** EvoTest decouples acting from adaptation using two distinct roles: an **Actor Agent** that plays a full episode and an **Evolver Agent** that improves the system between independent episodes. After each episode, the Evolver Agent analyzes the full transcript and proposes a revised configuration for the entire agentic system. This process of whole-system evolution involves:

1. Rewriting the guiding prompt to encode new strategies;
2. Updating a structured deployment-time memory with records of successful and failure actions;
3. Tuning decision-making hyperparameters like temperature and exploration strength;
4. Refining the tool-use routines that govern how and when memory or python code is accessed.

By evolving the agent configuration, EvoTest transforms the narrative of one episode into multi-faceted improvements for the next attempt, enabling a deeper form of learning than prior methods. We summarize our contributions as follows:

- **A Benchmark for Test-Time Learning:** We propose J-TTL, a benchmark using Jericho games to measure an agent's on-the-fly learning ability across a series of playthroughs of the same game.
- **A Test-time Learning Algorithm:** We propose EvoTest, an evolutionary agent learning framework that evolves the entire agentic system (policy, memory, tool-use routines, and hyperparameters) via transcript-level analysis without gradients or fine-tuning.
- **State-of-the-Art Empirical Results:** We demonstrate on the J-TTL benchmark that EvoTest shows a 38% improvement over the strongest prompt-evolution baseline and a 57% improvement over online RL, outperforming all strong reflection-based, memory-based, and gradient-based baselines on every game.

3 THE JERICHO TEST-TIME LEARNING (J-TTL) BENCHMARK

To systematically measure and drive progress in on-the-fly agent learning, we introduce the **Jericho Test-Time Learning (J-TTL)** benchmark. This benchmark is built upon the Jericho (Hausknecht et al., 2020)[1] suite of Interactive Fiction (IF) games. IF games are fully text-based simulation en-

[1]Jericho is available at https://github.com/Microsoft/jericho

vironments where an agent issues text commands to effect change in the environment and progress through a story. While the richness of these environments makes them a challenging testbed for AI, existing evaluation has primarily focused on single-episode performance or generalization across different games (Hausknecht et al., 2020; Gulcehre et al., 2020; Li et al., 2025). The J-TTL benchmark refocuses the evaluation on a different, critical axis: an agent's ability to learn and improve its strategy through repeated attempts at the same complex task within a single test session.

Datasets. We use publicly available Jericho games that vary in difficulty and puzzle structure, including *Detective*, *Library*, *Zork1*, *Zork3*, *Balances*, and *Temple*. Games are launched via Jericho with default scoring. Each episode is capped by a step limit ($T = 110$ unless stated otherwise).

The Jericho Game. We model a Jericho game (Hausknecht et al., 2020) as a Partially Observable Markov Decision Process (POMDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \Omega, T)$. Here, $\mathcal{S}$ is the latent state space, and $\mathcal{A}$ is the (infinite) combinatorial action space of natural language commands. At each step t, an agent in a latent state $s_t \in \mathcal{S}$ takes an action $a_t \in \mathcal{A}$, causing a transition to a new state $s_{t+1} \sim \mathcal{T}(\cdot \mid s_t, a_t)$ and yielding a scalar reward $r_t = \mathcal{R}(s_t, a_t)$. The agent does not observe the true state s_t but instead receives a textual observation $o_t \sim \Omega(\cdot \mid s_t)$. An episode is a trajectory of interactions with a finite horizon of T steps:

$$\tau^{(e)} \triangleq \left(o_1^{(e)}, a_1^{(e)}, r_1^{(e)}, \ldots, o_T^{(e)}, a_T^{(e)}, r_T^{(e)}\right). \tag{1}$$

The total return for an episode e is the sum of its rewards, $R(e) \triangleq \sum_{t=1}^{T} r_t^{(e)}$.

Test-Time Learning Process on Jericho. The J-TTL benchmark protocol consists of a **session** of K consecutive **episodes** played in a single game. After each episode concludes, the game environment is reset to its identical initial state, $s_0^{(e)} = s_{\text{init}}$ for all $e \in \{1, \ldots, K\}$. This ensures that any performance improvement is solely attributable to the agent's internal learning process.

Let the agent be parameterized by a set of learnable components $\theta \in \Theta$. In episode e, the agent's behavior is governed by a policy conditioned on its current learnable components, $\pi_{\theta^{(e)}}$.

The core of test-time learning lies in the update rule an agent applies between episodes. After completing episode e and collecting the trajectory data $\tau^{(e)}$, the agent updates its learnable components for the next episode:

$$\theta^{(e+1)} = U(\theta^{(e)}, \tau^{(e)}), \tag{2}$$

where $U : \Theta \times \mathcal{T}_{\text{hist}} \to \Theta$ is the agent's learning algorithm.

Evaluation Metrics. For a test-time learning session consisting of K episodes, we record total score for each episode, $R(e)$. This yields a performance sequence for the entire session:

$$\{R(1), R(2), \ldots, R(K)\}. \tag{3}$$

From this sequence, we derive two metrics:

- **Learning Curve.** A plot of the final return $R(e)$ against the episode index e. This curve provides a direct visualization of an agent's learning progress.
- **Area Under the Curve (AUC).** For quantitative comparison, we define the AUC as a normalized ratio. It measures the agent's total achieved score against the maximum possible score over the session:

$$\text{AUC} = \frac{\sum_{e=1}^{K} R(e)}{K \cdot R_{\max}}, \tag{4}$$

  where $R_{\max}$ is the maximum achievable score in a single episode of the game. This metric yields a value between 0 and 1, facilitating comparison across games with different scoring scales.

4 The EvoTest Framework

To address the challenges posed by the J-TTL benchmark, we introduce **EvoTest**, an evolutionary test-time learning framework, as illustrated in Figure 1. Unlike methods that perform gradient-based updates on model weights, EvoTest operates on a fixed, non-trainable backbone LLM. It achieves learning by evolving the agent's entire high-level configuration between episodes, leveraging the rich, narrative feedback from game transcripts rather than just sparse, scalar rewards.

4.1 THE AGENTIC CONFIGURATION

In the context of EvoTest, the general learnable components θ from the J-TTL formulation (Section 3) are instantiated as a holistic **agentic configuration**, denoted by $\chi \in \mathcal{X}$. This configuration is a tuple $\chi = (p, M, h, u)$ that defines the agent's complete operational strategy:

- **Policy Prompt (p):** A system prompt that provides high-level strategic guidance, heuristics, and behavioral guardrails to the backbone LLM.
- **Deployment-time Memory (M):** A structured, queryable database populated by the Evolver Agent after each episode. It stores the agent's prior experiences, effectively creating a persistent knowledge base. The memory is organized into distinct components, such as: (a) a *success memory* that logs state-action pairs which led to score increases (e.g., `(state_hash, action) -> score_delta`), and (b) a *failure memory* that records patterns associated with stalls or negative outcomes (e.g., repetitive action loops in a specific location). This allows the agent to recall specific, effective actions and avoid known pitfalls from past playthroughs. For a detailed breakdown of the memory's data structure and concrete examples of its contents, please see Appendix J.
- **Hyperparameters (h):** A set of parameters controlling the LLM's inference and the agent's decision-making, such as temperature, exploration strength, and stopping criteria.
- **Tool-Use Routines (u):** Active components executed at each decision-making step to operationalize knowledge and create useful state abstractions. These routines consist of two functions:
  - **Memory Interaction Logic:** A set of rules governing how to query the memory (M). Before deciding on an action, this routine might check if the current game state exists in memory. If a match is found in the success memory, the routine can inject the proven action into the LLM's prompt as a strong suggestion (e.g., "Hint: In this exact situation before, the action 'unlock door with key' worked well.").
  - **State Abstraction Logic:** An evolvable Python function—the *state extractor*—that processes the raw, verbose game history into a concise summary of progress. Instead of forcing the LLM to re-read the entire history, this tool parses the log for key environmental cues (e.g., the text `"The ancient scroll disintegrates, revealing a map."`) and returns a short, meaningful milestone string (e.g., `"Milestone: Found the map."`). This abstracted state is included in the prompt at every step, providing efficient situational awareness.

The agent's policy in episode e, $\pi_{\chi^{(e)}}$, is therefore a function of the fixed LLM's behavior as modulated by this entire configuration.

4.2 THE TWO-AGENT LEARNING LOOP

EvoTest operationalizes the test-time learning update rule, $\chi^{(e+1)} = U(\chi^{(e)}, \tau^{(e)})$, through a cooperative two-agent design that separates acting from adaptation.

1. The Actor Agent. For a given episode e, the Actor Agent is provided with a single, fixed configuration $\chi^{(e)}$. It plays through the episode by repeatedly querying the backbone LLM, conditioning its actions on the current observation o_t and any information retrieved from its memory $M^{(e)}$ as dictated by its tool-use routines $u^{(e)}$. The result of its playthrough is the trajectory $\tau^{(e)}$ and the final return $R(e)$.

2. The Evolver Agent. After the episode, the Evolver Agent takes the trajectory transcript $\tau^{(e)}$ and the parent configuration $\chi^{(e)}$ as input. It performs **whole-system evolution** by generating a set of new candidate child configurations, $C^{(e+1)} = \{\tilde{\chi}_1^{(e+1)}, \ldots, \tilde{\chi}_m^{(e+1)}\}$. This is the core learning step, where the Evolver uses an LLM to analyze the previous episode's trajectory and propose improvements to the agentic configuration. The evolutionary operators include:

- **Prompt Mutation:** The Evolver rewrites the policy prompt $p^{(e)}$ to create a new prompt $\tilde{p}$. It incorporates strategies that proved effective (e.g., adding "always examine objects before taking them") or adds explicit rules to prevent observed failures (e.g., "avoid repetitive navigation loops").
- **Memory Update:** The Evolver programmatically parses the transcript $\tau^{(e)}$ to identify and log important events. It records state-action pairs (o_t, a_t) that immediately preceded a score increase in a

"success" table. It also identifies sequences of interactions that led to no progress and records them in a "failure" table. This updated memory $M^{(e+1)}$ is then inherited by all child configurations.

- **Hyperparameter Tuning:** The Evolver proposes adjustments to the hyperparameters $h^{(e)}$ to create $\tilde{h}$. For example, if the transcript shows the agent was stuck in a repetitive loop, the Evolver might suggest increasing the LLM's 'temperature' to encourage more diverse actions.
- **Tool-Use Refinement:** The Evolver can modify the logic in $u^{(e)}$ to create $\tilde{u}$, changing when or how the agent consults its memory. For example, it might strengthen a rule in the prompt from a general suggestion to a direct instruction, such as telling the agent it must check its success memory before every action. The specification of these tools and the required JSON output format are detailed in the master prompt provided in Appendix H.

A child configuration $\tilde{\chi}$ is a new combination of these potentially mutated components, representing a new hypothesis for a more effective agent.

4.3 Configuration Selection via UCB

After the Evolver generates a set of new "child" configurations, EvoTest decide which one to use for the next episode. This creates a classic dilemma: should we *exploit* a configuration that has worked well in the past, or should we *explore* a new, untested one that might be even better?

To manage this trade-off, we select the next configuration, $\chi^{(e+1)}$, from a candidate pool containing the parent from the previous episode, $\chi^{(e)}$, and its new children, $C^{(e+1)}$. The selection is guided by the Upper Confidence Bound (UCB) algorithm, a strategy from multi-armed bandit theory (Vermorel & Mohri, 2005) designed to manage the exploration-exploitation dilemma. The rule selects the configuration that maximizes the following score:

$$\chi^{(e+1)} = \underset{\tilde{\chi} \in \{\chi^{(e)}\} \cup C^{(e+1)}}{\arg\max} \left(\hat{\mu}(\tilde{\chi}) + \beta \sqrt{\frac{\log N}{1 + n(\tilde{\chi})}} \right) \tag{5}$$

The score for each configuration $\tilde{\chi}$ is a sum of two parts:

- **Performance Term ($\hat{\mu}(\tilde{\chi})$):** This is simply the average score the configuration has achieved so far. It encourages us to re-use configurations that have a good track record.
- **Exploration Bonus ($\beta\sqrt{\dots}$):** This term gives a boost to configurations that are less certain about. $n(\tilde{\chi})$ is the number of times a configuration has been tried, N is the total number of episodes completed, and β is a hyperparameter controlling the exploration strength. The bonus is higher for configurations that have been tried fewer times ($n(\tilde{\chi})$ is small), making novel mutations attractive.

At the start of each new episode, the UCB algorithm selects a single configuration, which is then used for the entire duration of that episode. The episode's final score is then used to update the statistics for that chosen configuration. Crucially, this UCB approach also makes our learning process more stable and helps prevent sharp drops in performance. A simpler, greedy strategy might get fooled by a new configuration that gets a lucky high score and then repeatedly fails. UCB avoids this pitfall. Because the reliable parent configuration is always in the selection pool, if a new child performs poorly after its initial trial, its average score ($\hat{\mu}$) will drop. The UCB rule can then naturally "fall back" to the time-tested parent, which retains its high score. This acts as a safety net, preventing the system from getting stuck on a bad evolutionary path and ensuring a more consistent improvement.

5 Experiments

Our experiments are designed to answer three central research questions regarding test-time learning on our J-TTL benchmark:

- **RQ1:** Does test-time learning (TTL) lead to meaningful performance improvements on complex tasks compared to a non-learning agent?
- **RQ2:** Does our proposed EvoTest framework enable more effective test-time learning compared to existing methods, such as those based on memory, reflection, and prompt optimization?

- **RQ3:** How does the gradient-free, evolutionary approach of EvoTest compare to traditional gradient-based RL methods in the context of test-time learning?

5.1 SETUP

Backbone LLM. We use two powerful API models for the Actor Agent: the cost-effective `google/gemini-2.5-flash` and the highly capable `anthropic/claude-4-sonnet-20250522`. The Evolver Agent, which performs the most complex reasoning, is powered by `openai/o3-2025-04-16`. The fine-tuning baselines (SFT and GRPO) are implemented on `qwen/qwen3-32b`, a large open-source model. Experiments with more LLM backbones can be found in Appendix N

Baselines. As a foundational reference, a non-learning **Static** agent with a fixed configuration establishes zero-shot performance. The learning baselines are grouped into four main categories: (1) **Memory-based Methods**, which include (1a) **Memory**, which places the complete session transcript history into the context for in-context learning, automatically truncating the oldest parts if the context limit is exceeded, and (1b) **RAG**, which retrieves relevant snippets from past trajectories; (2) **Reflection-based Methods**, which learn by appending textual feedback to the prompt, including (2a) **Summary**, which uses an LLM to progressively summarize the entire history of all past transcripts and feeds this condensed summary into the context, and (2b) **Reflexion** (Shinn et al., 2023), which generates structured textual self-reflections after each episode to critique performance and guide the next attempt; (3) **Automated Prompt Optimization Methods**, which iteratively refine the guiding prompt, including (3a) **TextGrad** (Yuksekgonul et al., 2024), where after each episode, an optimizer LLM analyzes the trajectory to generate a "textual gradient"—a critique describing how to improve the prompt—which is then applied for refinement, and two evolutionary approaches, (3b) **Promptbreeder** (Fernando et al., 2023) and (3c) **EvoPrompt** (Guo et al., 2024), which evolve a population of prompts; and (4) **Weight-Update Methods**, which contrast with our gradient-free approach by performing online fine-tuning, including (4a) **SFT (online)**, which performs Supervised Fine-Tuning on the actor model after each episode using the state-action pairs collected from the trajectory, and (4b) **GRPO (online)** (Shao et al., 2024), which applies gradient-based Reinforcement Learning policy updates to the model using the scalar rewards collected during the episode. All methods are evaluated under the same step budget and use the same backbone LLMs for their respective roles to ensure a fair comparison. The detailed introduction can be found in appendix C, D.

5.2 RESULTS

Table 1 presents the AUC scores for all methods across the six selected Jericho games, while the learning curves in Figure 2 illustrate the per-episode performance progression. We analyze these results through the lens of our three research questions.

RQ1: Test-Time Learning is Effective. The results provide a clear affirmative answer. Across all games, every learning-based method achieves a higher average AUC score than the **Static** baseline (Table 1). The learning curves (Figure 2) further confirm this, showing that methods capable of learning from experience consistently exhibit upward-trending scores, whereas the Static agent's performance remains flat. This demonstrates that even with just a few attempts, test-time learning is a valid and effective paradigm for improving agent performance on complex, long-horizon tasks.

RQ2: EvoTest Outperforms Existing Adaptation Methods. EvoTest consistently and substantially outperforms all other gradient-free baselines. In Table 1, EvoTest achieves the highest AUC score on all six games, with an average score of 0.47/0.50—a significant improvement over the next best baseline, EvoPrompt (0.34/0.36). The learning curves reveal not just higher final scores but a steeper rate of improvement, indicating more efficient learning.

The key insight lies in the limitation of single-channel adaptation. **Memory** and **RAG** provide raw information but offer no strategic guidance, leading to a low performance ceiling. **Reflexion** and other prompt-focused optimizers like **Promptbreeder** perform better by refining strategy, but they are constrained to a single axis: the prompt. An agent can have a perfect prompt but still fail due to poor exploration (e.g., low temperature) or inefficient use of its knowledge. EvoTest's strength is its **holistic, whole-system evolution**. By concurrently optimizing the prompt, memory-access routines, and decision hyperparameters, it discovers and resolves complex performance bottlenecks that single-channel adaptations cannot. For example, it can learn to increase exploration temperature in early episodes and simultaneously add a new strategic heuristic to its prompt based on its findings, a multi-faceted adaptation that other methods are incapable of, as detailed and plotted in Appendix M.

RQ3: Evolutionary Adaptation is More Data-Efficient than RL at Test Time. The comparison between EvoTest and the weight-update methods clearly favors the evolutionary approach in this setting. EvoTest's average AUC (0.47/0.50) is substantially higher than that of **GRPO (online)** (0.30), the gradient-based RL baseline. This improvement indicates that EvoTest successfully addresses a fundamental challenge in test-time learning: extreme data scarcity.

Moreover, EvoTest alleviate traditional RL's reliance on scalar rewards for credit assignment. In sparse-reward environments like Jericho, a single episode provides a noisy and insufficient signal for effective gradient updates. It is an inefficient way to learn from one complex success or failure. In contrast, EvoTest bypasses this issue by leveraging the **entire episode transcript as a rich, narrative feedback signal**. The Evolver Agent performs credit assignment through semantic analysis of the game's story, identifying causal chains of failure (e.g., "the agent got stuck in a loop here") and success. This allows it to make explicit, targeted, and structural edits to the agent's configuration. In essence, EvoTest shifts from credit assignment via backpropagation to **credit assignment via narrative analysis**, a more data-efficient mechanism for learning from a single experience.

5.3 Model Analysis

Ablation on Key Components. Our ablation study reveals the distinct contributions of each component in the EvoTest framework. The AUC scores in Table 3 quantify the overall impact, showing that removing any component degrades performance. The largest performance drop occurs when removing prompt evolution (*w/o Prompt*), confirming that evolving the high-level policy is the primary driver of strategic adaptation.

The learning curves in Figure 3 offer deeper insight into the *dynamics* of these failures, particularly for the UCB ablation. While removing UCB also causes a significant drop in AUC, the curve reveals a more nuanced issue: instability. The *w/o UCB* agent, which uses a greedy selection strategy, is prone to catastrophic drops in performance. This happens when the agent over-commits to a high-risk mutation that achieved a lucky high score in one episode but is not robust. Without UCB's exploration mechanism—which encourages revisiting more reliable past configurations—the agent gets stuck on a suboptimal evolutionary path and is unable to correct its course after a poor choice. In contrast, the full EvoTest model leverages UCB to maintain a stable learning trajectory.

Efficiency Analysis. For learning at test-time, the update step between episodes is a major bottleneck. Our experiments show a clear divide in practicality between different approaches, as detailed in Table 2. Weight-update methods like online RL are not practical for this setting. A fine-tuning pass on one episode's data took 5 to 10 minutes on 4 H100 GPUs. This is not just slow; it demands expensive hardware, making it a non-starter for a system that needs to learn at test time. In contrast, EvoTest and other gradient-free methods operate on a different timescale. Instead of a costly training

run, our learning step is a single API call to an LLM, which takes about 20-30 seconds. We provide a more formal analysis of the computational complexity of EvoTest in Appendix E

Impact of the LLM of Evolver Agent. To assess the sensitivity of our framework to the Evolver's reasoning capabilities, we conducted an ablation study on its underlying LLM (Table 4). The results reveal a clear correlation between model scale and agent performance; more powerful models like `openai/o3` consistently yield higher scores, likely due to their superior ability to distill complex strategic insights from raw episode transcripts. Notably, even with a significantly smaller model such as `qwen3-8b`, performance remains substantially above the non-learning *Static* baseline. This finding demonstrates the robustness of the EvoTest framework: while a more capable Evolver LLM acts as a performance amplifier, the fundamental act-evolve loop is effective in its own right.

Ablation on the Structure of Prompt Evolution. To further isolate the contribution of our prompt-evolving logic, we create a baseline, "EvoTest (w/ Simple Mutation)," which replaces our detailed Evolver prompt with a generic instruction to simply "analyze the trajectory and generate an improved prompt." The results in Table 5 show that while this simpler mutation improves over no prompt evolution, it significantly underperforms our system, with the AUC score on Detective dropping from 0.94 to 0.65. This demonstrates that the performance gains are not just from evolving the prompt, but from the Evolver's sophisticated analysis, which is a core contribution of our framework.

A Fairer Comparison with Fine-Tuning Methods. In this setup, we normalize the underlying model capabilities by using `qwen/qwen3-32b` as the backbone for all compared methods. As shown in Table 6, we evaluate two versions of our method: one using `qwen/qwen3-32b` for both the Actor and Evolver, and a second version that pairs the `qwen/qwen3-32b` Actor with the stronger `openai/o3` Evolver. The results confirm that even when both components use `qwen/qwen3-32b`, EvoTest (Avg. AUC 0.35) outperforms the strongest weight-update baseline, GRPO (0.31). Furthermore, the performance leap when using a more capable Evolver (Avg. AUC 0.40) underscores the impact of the optimizer's reasoning ability within our framework.

6 Conclusion

We introduce the J-TTL benchmark to measure test time agent learning and proposed EvoTest, a novel evolutionary framework that improves agentic systems at test-time without gradients. By analyzing entire episode transcripts, EvoTest evolves the complete agent configuration—policy, memory, and hyperparameters—to rapidly adapt. Our experiments show EvoTest significantly outperforms strong baselines, including reflection, prompt optimization, and online fine-tuning. Its strength lies in using rich, narrative feedback for credit assignment, a far more data-efficient paradigm than relying on sparse rewards. This work provides a concrete step toward building truly autonomous agents that learn and self-improve from experience. In the future, it is worthwhile building self-evolving agents for extremely long-horizon tasks (Liu et al., 2026)